

Initial C++ Class Development and Module Ideas

Class Timer:

Timer is a utility class for making precise timing measurements in the program. It will start primarily as a developer tool, but as the project evolves it could later become something users can interact with in the game...(if our company decides that's useful). For now, it allows code to start and stop named timers so the team can monitor performance throughout development. In addition, Timer will store timing results for each name (last duration, count, average/min/max) so different parts of the program can be compared and performance issues are easy to spot.

Similar Classes:

- `std::chrono` (especially `std::chrono::steady_clock`, `duration`, `time_point`)
- `std::unordered_map` (store timers keyed by name)
- `std::string` / `std::string_view` (names)
- `std::optional` (optional currently running state, if used)
- RAI pattern (optional "ScopedTimer" helper)

Key Functions:

- `Start(name)`: begins timing a named section
- `Stop(name)`: ends timing and records the elapsed duration for that name
- `Reset(name)`: clears stored results for one timer name
- `ResetAll()`: clears all timers and measurements
- `Last(name)`: most recent recorded duration
- `Min(name)`, `Max(name)`: fastest/slowest recorded run
- `Average(name)`: average duration across all recorded runs
- `Names()`: returns the set of timer names currently tracked
- `Report()`: returns a readable summary (useful for debug output).

Error Conditions:

- Calling `Stop(name)` when that timer isn't currently running (never started, already stopped, or name is wrong)
 - (1) Programmer error: handled with an assert in debug builds
- Calling `Start(name)` when that timer is already running
 - (1) Programmer error: handled with an assert in debug builds
- Asking for statistics when there are no completed measurements (or the name hasn't been used yet)

- (3) User/usage case: I provide HasData(name) to check first; if empty, stat functions will return a default

Expected Challenges/ topics to learn

- Using std::chrono correctly (steady clock, durations/time points).
- Managing “running vs. stopped” state cleanly for each named timer.
- Formatting a simple, consistent Report() output.

Groups to coordinate with:

- OutputManager: Timer reports can be printed through the logging system; we may align formatting and optional timestamps/verbosity levels.
- ActionLog / ReplayDriver: can use Timer to measure replay speed, action processing time, and tick timing consistency.
- World / Agent teams: agree on common timer names (ex world_update, agent_step, render, network_sync) so performance reports are comparable across modules.

Class ReplayDriver:

1. ReplayDriver controls replay mode for the game by reading from the action log and selecting the previously recorded action to control all agents in our world. During the replay, the agents will have no control over decisions, instead ReplayDriver will assign actions based on the recorded sequence and ensure that the simulation executes exactly like the original execution. This can be useful for debugging, testing, and data analysis.
2. Std::iterator, std::ostream, std::queue
3. Potential functions:
 - a. startR(): starts the replay mode and stops the agents in the simulation to take full control over their actions
 - b. action(): the replay proceeds by one recorded action from the action log and it is applied to the correct agent
 - c. sendAction(): send the recorded action to the correct agent
 - d. isFinished(): indicates if they replay has completed
 - e. reset(): resets the replay to the beginning of the action log
 - f. pause(): option to pause the replay in the middle of the sequence
 - g. resume(): option to resume the replay after being paused
4. Error conditions:
 - a. Invalid world or action log: a programmer error; uses an assert to ensure that when world/action log is initialized that it is not a nullptr

- b. Invalid replay transition: a programmer error; uses an assert to enforce correct transitions
 - c. Replay interrupted or paused by user: user-controlled condition that returns a special state
- 5. I expect to have a challenge with the replay copying exact actions with the correct timing since this requires coordination with the action log timing. This can be complicated. I also expect to struggle with switching between normal simulation mode and replay mode since I need to be careful with who owns control over the agent's decisions all the time.
- 6. I will need coordination with the action log because this is the primary data source for replaying. I will also need coordination with an agent class to control their decision making and with a world class to control the simulation state.

The data analytics module can provide tools to collect and analyze data to support behavioral analysis and optimize performance. The action log records actions and timing information to enable the replay driver. The data log and timer classes track performance and numerical data, which allows analysis and comparisons within the simulation. The output manager allows the groups to observe system behaviors in distinct detail. Together, these components offer a combined analytics framework that can integrate with the rest of the system to provide relevant insight without disrupting the normal execution of the simulation.

Class DataLog:

1. The goal of the DataLog class is to track a series of useful data values over time. Its functionality is to return statistics such as the mean, median, min, max, or whichever other statistics that are valuable.
2. Some classes from the standard library are `std::vector` or `std::priority_queue` to store the data and `std::algorithm` to find the minimum, maximum, and other characteristics of the data.
3. Some key functions I plan to implement are `Add()` to add data, and `Mean()`, `Median()`, `Min()`, and `Max()` for returning statistics. More functions may be added later for other useful statistics.
4. If there are invalid inputs to functions, that is a programmer error and will be caught by an assert. There might be no data to return statistics on so that is a user error and will return a special condition.
5. There might be a large amount of data, so an expected challenge is to ensure you store and compute the data efficiently.

6. The web interface group will get statistics from this class so we will likely coordinate with them.

Class ActionLog:

1. ActionLog will track all the actions/moves made by all the agents within the current world. Additionally, all the actions/moves will have a time stamp and will track timings in between moves allowing replay, analysis, and debugging of agent behaviors.
2. Classes I need to be familiar with
 - a. Any container class such as a map, vector, or queue
 - b. Time Handling classes like Chrono or the timer class mentioned earlier
3. Key functions that will be implemented: recordAction, getActions, save, and load
4. Errors
 - a. Invalid timestamps (e.g., decreasing timestamps without proper justification)
 - b. Invalid AgentID (out of valid range)
5. My main expected challenge will be creating the action log to handle the companies made agents. Specifically, communication across the teams in order for me to create a functional and useful log.
6. Timer and Agents classes

Class OutputManager:

1. OutputManager is a simple logging utility used to print messages during program execution. It allows developers to control how much information is displayed by supporting multiple log levels. This helps with debugging, monitoring agent behavior, and understanding system state during runtime.
2. Responsibilities
 - a. Provide a unified logging interface across the entire project
 - b. Support multiple log levels to control output verbosity
 - c. Optionally include timestamps and metadata with log message
 - d. Allow logs to be routed to different outputs (console, file, buffer)
 - e. Be safe and efficient enough to use in performance-critical code
3. Log Levels
 - a. Silent – No output
 - b. Normal – Important system messages
 - c. Verbose – detailed runtime information
 - d. Debug – Fine grained debugging output
4. Project Relevance

- a. The project involves many interacting systems. A shared logging system makes it easier to debug problems, trace actions, and understand how the system behaves as it grows in complexity

Our group's vision for the main module is a high-interaction based simulated world. We are leaning towards a more city based simulated world, but we are also discussing the possibility of it being a more medieval type of civilization. AI will be used for our agents alongside our classes we will be creating within our group. More discussion within the entire company is needed to finalize our theme, but our general concept of an AI based civilization builder has been agreed upon.